

100 задач на структуры, указатели и методы в Go

1.

Определите структуру `User` с полями `ID int`, `Name string`, `Age int`. Напишите функцию `NewUser(id int, name string, age int) User`, которая создает и возвращает пользователя.

2.

Определите структуру `Point` с координатами `X int` и `Y int`. Напишите функцию `NewPoint(x, y int) Point`.

3.

Определите структуру `Rectangle` с полями `Width int` и `Height int`. Напишите функцию `Area(r Rectangle) int`.

4.

Для структуры `Point` напишите функцию `IsOrigin(p Point) bool`, которая возвращает `true`, если обе координаты равны нулю.

5.

Для структуры `User` напишите функцию `Older(a, b User) User`, которая возвращает более старшего пользователя. При одинаковом возрасте возвращайте `a`.

6.

Для структуры `User` добавьте поле `Email string`. Напишите функцию `WithEmail(u User, email string) User`, которая возвращает измененную копию пользователя и не меняет исходный `u`.

7.

Определите структуру `Pair` с полями `A int` и `B int`. Напишите функцию `SwapPair(p Pair) Pair`, возвращающую структуру с переставленными значениями.

8.

Для двух точек `Point` напишите функцию `ManhattanDistance(a, b Point) int`.

9.

Определите структуру `Range` с полями `From int` и `To int`. Напишите функцию `NormalizeRange(r Range) Range`, которая гарантирует `From <= To`.

10.

Напишите функцию `MovePoint(p Point, dx, dy int) Point`, которая возвращает новую точку, не изменяя исходную.

11.

Определите структуру `Student` с полями `Name string` и `Grades []int`. Напишите функцию `AverageGrade(s Student) float64`. Для пустого списка оценок возвращайте `0`.

12.

Определите структуру `Product` с полями `Name string` и `Price int`. Напишите функцию `Discounted(p Product, percent int) Product`, возвращающую копию товара со сниженной ценой. Процент считать в диапазоне от 0 до 100.

13.

Определите структуру `Employee` с полями `Name string`, `Rate int` и `Hours int`. Напишите функцию `Salary(e Employee) int`.

14.

Определите структуру `ClockTime` с полями `Hour int` и `Minute int`. Напишите функцию `NormalizeTime(t ClockTime) ClockTime`, которая переносит лишние минуты в часы и приводит время к диапазону суток.

15.

Определите структуру `Date` с полями `Year, Month, Day int`. Напишите функцию `CompareDate(a, b Date) int`, возвращающую `-1`, `0` или `1`.

16.

Напишите функцию `FindUserByID(users []User, id int) (User, bool)`, которая ищет пользователя по ID.

17.

Напишите функцию `Adults(users []User) []User`, возвращающую только пользователей с `Age >= 18`. Исходный слайс не изменять.

18.

Напишите функцию `IndexUsers(users []User) map[int]User`. Если ID повторяется, в мапе должен остаться последний пользователь с этим ID.

19.

Добавьте в `User` поле `City string`. Напишите функцию `GroupUsersByCity(users []User) map[string][]User`, сохраняя исходный порядок пользователей внутри каждой группы.

20.

Определите структуру `Employee` с полями `Name string` и `Salary int`. Напишите функцию `HighestPaid(items []Employee) (Employee, bool)`.

21.

Определите структуру `Task` с полем `Status string`. Напишите функцию `CountByStatus(tasks []Task) map[string]int`.

22.

Определите `OrderItem{Name string; Price int; Quantity int}` и `Order{ID int; Items []OrderItem}`. Напишите функцию `OrderTotal(o Order) int`.

23.

Определите `Cart{Items []OrderItem}`. Напишите функцию `CartItemsCount(c Cart) int`, которая считает общее количество единиц товаров с учетом `Quantity`.

24.

Напишите функцию `BestStudent(students []Student) (Student, bool)`, выбирающую студента с максимальным средним баллом. При равенстве выбирайте первого.

25.

Определите `Address{City, Street string; House int}` и `Person{Name string; Address Address}`. Напишите функцию `SameCity(a, b Person) bool`.

26.

Напишите функцию `UpdateUserName(users []User, id int, name string) bool`, которая изменяет элемент с нужным ID прямо в переданном слайсе и возвращает, найден ли пользователь.

27.

Напишите функцию `RemoveUserByID(users []User, id int) []User`, удаляющую первое совпадение и сохраняющую порядок остальных элементов.

28.

Напишите функцию `UniqueUsersByID(users []User) []User`, сохраняющую только первое появление каждого ID.

29.

Определите `Profile{Name string; Tags []string}`. Напишите функцию `CloneProfile(p Profile) Profile`, которая создает независимую копию структуры и слайса `Tags`.

30.

Определите `Settings{Flags map[string]bool; Values []int}`. Напишите функцию `CloneSettings(s Settings) Settings`, которая делает глубокую копию карты и слайса.

31.

Напишите функцию `Inc(v *int)`, увеличивающую значение по указателю на 1.

32.

Напишите функцию `Swap(a, b *int)`, меняющую местами два значения.

33.

Напишите функцию `SetZero(v *int)`, записывающую по указателю ноль.

34.

Напишите функцию `AbsInPlace(v *int)`, заменяющую отрицательное значение на положительное.

35.

Напишите функцию `ClampInPlace(v *int, min, max int)`, ограничивающую значение заданным диапазоном.

36.

Напишите функцию `OrderTwo(a, b *int)`, после которой `*a <= *b`.

37.

Для структуры `Pair` напишите функцию `SwapPairInPlace(p *Pair)`, меняющую поля `A` и `B` местами.

38.

Для `Point` напишите функцию `MovePointInPlace(p *Point, dx, dy int)`.

39.

Для `User` напишите функцию `RenameUser(u *User, name string)`.

40.

Для `User` напишите функцию `Birthday(u *User)`, увеличивающую возраст на 1.

41.

Для `Product` напишите функцию `ApplyDiscount(p *Product, percent int)`, меняющую цену на месте.

42.

Напишите функцию `SetPositive(v *int, value int) bool`: если `value > 0`, запишите его в `*v` и верните `true`, иначе ничего не меняйте.

43.

Напишите функцию `SetIfGreater(dst *int, candidate int) bool`, которая обновляет значение только если кандидат больше текущего.

44.

Напишите функцию `ResetUser(u *User)`, присваивающую структуре нулевое значение `User`.

45.

Напишите функцию `CopyInt(dst, src *int) bool`. Если любой указатель равен `nil`, ничего не делайте и верните `false`; иначе скопируйте значение и верните `true`.

46.

Напишите функцию `CloneUser(u *User) *User`. Для `nil` возвращайте `nil`, иначе создавайте новый независимый объект с теми же полями.

47.

Напишите функцию `SwapUsers(a, b *User) bool`. Если любой указатель `nil`, ничего не меняйте и верните `false`.

48.

Напишите функцию `FindUserPtr(users []User, id int) *User`, возвращающую указатель именно на элемент исходного слайса, чтобы изменение через указатель меняло `users`.

49.

Дана `map[int]User`. Напишите функцию `RenameMapUser(users map[int]User, id int, name string) bool`, корректно изменяющую имя пользователя в мапе через схему read-modify-write.

50.

Определите `UserProfile{Name string; Address *Address}`. Напишите функцию `SetCity(p *UserProfile, city string) bool`. Если `p == nil` или `p.Address == nil`, возвращайте `false`.

51.

Определите `Profile{Name string; Tags []string}`. Напишите `AddUniqueTag(p *Profile, tag string) bool`, добавляющую тег только если его еще нет.

52.

Определите `Config{Labels map[string]string; Ports []int}`. Напишите `DeepCloneConfig(c *Config) *Config`, чтобы изменения копии не влияли на исходную структуру, ее мапу и слайс.

53.

Определите односвязный узел `Node{Value int; Next *Node}`. Напишите функцию `ListSum(head *Node) int`.

54.

Для `Node` напишите функцию `AppendNode(head *Node, value int) *Node`, которая добавляет новый узел в конец и корректно работает с пустым списком.

55.

Для `Node` напишите функцию `ReverseList(head *Node) *Node`, разворачивающую односвязный список без создания новых узлов.

56.

Для `Point` реализуйте метод `ManhattanTo(other Point) int` с `value receiver`.

57.

Для `Rectangle` реализуйте методы `Area() int` и `Perimeter() int` с value receiver.

58.

Для `User` реализуйте метод `IsAdult() bool`.

59.

Определите `Money{Amount int64; Currency string}`. Реализуйте метод `Add(other Money) (Money, bool)`: суммы можно складывать только при одинаковой валюте.

60.

Для `Date` реализуйте метод `IsBefore(other Date) bool`.

61.

Для `Range` реализуйте метод `Contains(v int) bool`.

62.

Для нормализованного `Range` реализуйте метод `Length() int`, считающий количество целых значений в диапазоне включительно.

63.

Определите `Counter{value int}`. Реализуйте метод `Value() int`, который только читает состояние и использует value receiver.

64.

Для `Order` реализуйте метод `Total() int`, суммирующий стоимость всех позиций.

65.

Определите `Circle{Radius float64}`. Реализуйте методы `Area() float64` и `Diameter() float64`. Для площади используйте `math.Pi`.

66.

Для `Counter` реализуйте метод `Inc()` с pointer receiver.

67.

Для `Counter` реализуйте метод `Add(delta int)` с pointer receiver.

68.

Для `Counter` реализуйте метод `Reset()` с pointer receiver.

69.

Определите `BankAccount{Balance int64}`. Реализуйте метод `Deposit(amount int64) bool`: отрицательные и нулевые суммы отклонять.

70.

Для `BankAccount` реализуйте метод `Withdraw(amount int64) bool`: баланс не должен становиться отрицательным.

71.

Для `User` реализуйте метод `Rename(name string) bool`: пустое имя отклонять, корректное имя записывать в структуру.

72.

Для `Profile` реализуйте метод `AddTag(tag string) bool`, добавляющий только уникальный непустой тег.

73.

Определите `Playlist{Tracks []string}`. Реализуйте методы `Add(track string)`, `Remove(track string) bool` и `Len() int`.

74.

Определите `Stack{items []int}`. Реализуйте методы `Push(v int)`, `Pop() (int, bool)` и `Len() int`.

75.

Определите `Queue{items []string}`. Реализуйте методы `Enqueue(v string)`, `Dequeue() (string, bool)` и `Len() int`.

76.

Определите `Inventory{stock map[string]int}`. Реализуйте методы `Add(name string, count int) bool` и `Count(name string) int`. Нулевое значение `Inventory` должно корректно инициализироваться при первом добавлении.

77.

Для `Inventory` реализуйте метод `Take(name string, count int) bool`, который не допускает отрицательного остатка и удаляет ключ при нулевом остатке.

78.

Для `Inventory` реализуйте метод `TotalUnits() int`, возвращающий суммарное количество всех товаров.

79.

Определите `Todo{ID int; Text string; Done bool}` и `TodoList{items []Todo}`. Реализуйте методы `Add(text string) int`, `Complete(id int) bool` и `Delete(id int) bool`.

80.

Определите `Contact{Name, Phone string}` и `AddressBook{contacts []Contact}`. Реализуйте методы `Add(c Contact)`, `FindByName(name string) (Contact, bool)` и `RemoveByName(name string) bool`.

81.

Определите `Book{ID int; Title string; Borrowed bool}` и `Library{books map[int]Book}`. Реализуйте `AddBook`, `Borrow(id int) bool` и `Return(id int) bool`.

82.

Определите `Wallet{Balance int64}`. Реализуйте функцию `Transfer(from, to *Wallet, amount int64) bool`: при любой ошибке оба кошелька должны остаться без изменений.

83.

Определите `Player{X, Y int}`. Реализуйте метод `Move(dx, dy, minX, maxX, minY, maxY int)`, который перемещает игрока, но не выпускает координаты за границы поля.

84.

Определите `Thermostat{Temperature int; Min int; Max int}`. Реализуйте метод `SetTemperature(v int)`, который ограничивает температуру диапазоном `Min..Max`.

85.

Определите `CartLine{ProductID int; Price int; Quantity int}` и `ShoppingCart{Items []CartLine}`. Реализуйте `Add(productID, price, quantity int) bool`: одинаковые товары объединять по `ProductID`.

86.

Для `ShoppingCart` реализуйте методы `Remove(productID int) bool` и `Total() int`.

87.

Определите `UserRegistry{users map[int]User}`. Реализуйте методы `Add(u User) bool`, `Find(id int) (User, bool)` и `Rename(id int, name string) bool`. Повторный ID добавлять нельзя.

88.

Определите `PlayerScore{Name string; Points int}` и `ScoreBoard{scores map[string]int}`. Реализуйте методы `AddPoints(name string, delta int)` и `Leader() (PlayerScore, bool)`. При равенстве очков выбирайте лексикографически меньшее имя.

89.

Определите `Reservation{seats []bool}`. Реализуйте методы `Reserve(index int) bool`, `Cancel(index int) bool` и `FreeCount() int`.

90.

Определите `StringCache{data map[string]string}`. Реализуйте методы `Put(key, value string)`, `Get(key string) (string, bool)` и `Delete(key string) bool`. Нулевое значение структуры должно быть пригодным к использованию.

91.

Реализуйте две функции: `RenameCopy(u User, name string) User` и `RenameInPlace(u *User, name string)`. Первая должна возвращать измененную копию, вторая - менять исходный объект.

92.

Определите `Report{Name string; Values []int}`. Реализуйте метод `Clone() Report`, создающий глубокую копию. После изменения `clone.Values` исходный `Report` не должен меняться.

93.

Определите `Catalog{Prices map[string]int}`. Реализуйте метод `Clone() Catalog` с полной независимостью внутренней карты.

94.

Определите `Profile{Name string; Tags []string; Scores map[string]int}`. Реализуйте метод `Equal(other Profile) bool` без `reflect.DeepEqual`; порядок тегов должен быть значимым.

95.

Определите `OptionalCounter{Value int}`. Реализуйте метод `IsZero() bool` на `*OptionalCounter`, который безопасно работает даже при `nil receiver` и для `nil` возвращает `true`.

96.

Для односвязного списка `Node` реализуйте функцию `InsertAfter(node *Node, value int) bool`. Для `nil` возвращайте `false`, иначе вставляйте новый узел сразу после переданного.

97.

Определите бинарное дерево `TreeNode{Value int; Left, Right *TreeNode}`. Напишите функцию `TreeSum(root *TreeNode) int`.

98.

Для `TreeNode` напишите функцию `TreeDepth(root *TreeNode) int`, где глубина пустого дерева равна 0, а одного узла - 1.

99.

Определите `Repository{users []*User}`. Реализуйте методы `Add(u *User) bool`, `Find(id int) *User` и `Delete(id int) bool`. Не допускайте `nil`, повторных ID и случайного копирования объектов при поиске.

100.

Финальная задача. Определите `Account{ID int; Balance int64}` и `AccountStore{accounts map[int]*Account}`. Реализуйте методы `Create(id int) bool`, `Deposit(id int, amount int64) bool`, `Transfer(fromID, toID int, amount int64) bool` и `Snapshot() map[int]Account`. Любая неуспешная операция не должна частично менять состояние, а `Snapshot` должен возвращать независимую копию текущих аккаунтов.